Open-Source, Edge-Optimized Corporate Credit Decisioning: A Zero-Trust, 24GB VRAM Architecture

The Indian corporate lending ecosystem is currently paralyzed by a phenomenon characterized as the "Data Paradox" [1]. Financial institutions possess access to unprecedented, exponentially growing volumes of intelligence—ranging from structured Goods and Services Tax (GST) returns and granular bank statements to highly unstructured board meeting minutes, annual reports, and external litigation records [1]. Yet, the processing and synthesis of a single mid-sized corporate loan application routinely extends into weeks due to structurally deficient manual methodologies [1]. In response to the mandate to construct an AI-powered Credit Decisioning Engine capable of automating the end-to-end preparation of a Comprehensive Credit Appraisal Memo (CAM), early architectural blueprints proposed the utilization of industrial-grade foundation models and massive cloud-tethered data lakes [1].

However, relying on closed-source, cloud-dependent architectures introduces severe regulatory vulnerabilities, data privacy risks, and unsustainable operating expenditures. Proprietary models obscure their training data and inference mechanisms, violating the strict algorithmic auditability required by modern financial regulatory frameworks [1]. Consequently, this report outlines a radical architectural pivot: the design and deployment of a 100% open-source, edge-optimized "Intelli-Credit" decisioning engine.

The ensuing microservices architecture is meticulously engineered to execute flawlessly within a strictly bounded computational budget—a single 24 GB VRAM GPU (e.g., NVIDIA RTX 4090 or AMD equivalent) paired with 64 GB of DDR5 system memory [2, 3, 4]. By replacing monolithic proprietary dependencies with highly quantized local models, dynamic Low-Rank Adaptation (LoRA) swapping, and advanced open-source orchestration frameworks, the system achieves state-of-the-art analytical accuracy while guaranteeing absolute data sovereignty, continuous offline availability, and operational efficiency [2, 3].

Pillar I: The Data Substrate Pivot (Deconstructing the Proprietary Lakehouse)

Enterprise data architectures frequently default to vendor-locked ecosystems like Databricks and its associated Medallion (Bronze/Silver/Gold) framework for data ingestion, normalization, and analytical processing [5]. While highly capable in distributed cloud environments, Databricks is a proprietary, managed service that incurs heavy computational overhead and recurring licensing costs, rendering it fundamentally incompatible with an isolated, locally hosted, zero-trust operational mandate [5, 6]. To achieve a fully open-source, lightweight data substrate that does not overwhelm the 64 GB DDR5 RAM limitation, the architecture

comprehensively abandons Databricks in favor of a localized, composable Lakehouse stack combining DuckDB, LanceDB, and Apache Iceberg [7, 8].

The transition to a localized stack begins with the deployment of DuckDB as the primary analytical engine. DuckDB operates as an embedded, in-process SQL online analytical processing (OLAP) database [9]. Unlike legacy big data frameworks such as Apache Spark, which rely on the Java Virtual Machine (JVM) and suffer from severe memory bloat and garbage collection latency, DuckDB requires zero server management and executes vectorized SQL queries directly on raw data files within the host process [9, 10]. By leveraging zero-copy execution methodologies, DuckDB queries Pandas and Apache Arrow dataframes directly in system memory with negligible latency, completely eliminating the data serialization bottlenecks that historically crippled local analytics [10]. This vectorized execution maximizes CPU cache locality, allowing complex aggregations over millions of rows of GSTIN transaction data to execute in milliseconds without spilling over into swap memory [10].

To maintain the structural integrity of the Medallion architecture—ensuring atomic updates, time travel, and robust state management between raw unstructured ingestion and clean analytical tables—the system utilizes the Apache Iceberg open table format [8]. Iceberg abstracts the underlying storage (which can be handled locally via an S3-compatible object store like MinIO) and provides robust transaction logging and schema evolution [6, 11]. This allows the local system to handle concurrent reads and writes from multiple autonomous agents simultaneously without encountering data corruption or read inconsistencies [12]. By consolidating metadata into a standard SQL database structure, the architecture eliminates the need for heavyweight, standalone catalog services [13].

Because the Intelli-Credit engine relies heavily on unstructured semantic data extracted from PDFs alongside tabular financial metrics, LanceDB is integrated to replace proprietary vector databases [8]. LanceDB is an open-source, developer-friendly database purpose-built for AI workloads, capable of storing both tabular metadata and high-dimensional vector embeddings natively within the Lance columnar format [14]. The integration of the Lance x DuckDB extension is a critical architectural advantage; it allows the system to perform vector similarity searches and complex SQL joins simultaneously, treating semantic retrieval as a standard SQL operator [7]. This unified data plane means that an autonomous agent can query for "litigation events related to Promoter X" (a semantic vector search) and seamlessly join the results with "transactions exceeding ₹1,000,000" (a structured SQL query) entirely within DuckDB, drastically reducing the memory footprint and operational complexity of the pipeline [7].

Architectural Component	**Proprietary Cloud Paradigm**	**Localized Open-Source Paradigm**	**Edge-Optimization Impact (64 GB RAM Budget)**
Compute Engine	Databricks (Apache	DuckDB	Eliminates JVM

	Spark) [5]	(Embedded OLAP) [9]	overhead; executes queries via zero-copy Arrow memory mapping, saving >12 GB RAM [10].
Storage Management	Unity Catalog (Proprietary) [15]	Apache Iceberg / MinIO [8]	Provides ACID transactions and schema evolution purely via open table formats on local disk [6].
Vector Retrieval	Pinecone / Weaviate (SaaS)	LanceDB [14]	Fuses multimodal vector storage directly with DuckDB via the Lance columnar format, eliminating network latency [7].

Pillar II: Visual Document Understanding and the GLM-OCR Paradigm

The foundational bottleneck in corporate credit appraisal is the ingestion and precise extraction of high-fidelity financial commitments, contingent liabilities, and granular statistical data from unstructured, degraded, and poorly scanned Indian-context PDFs [1]. The initial architectural proposal mandated the use of Unisound U1-OCR, citing its ability to achieve "OCR 3.0" capabilities through a proprietary coordinate-text-semantics framework [1]. While corporate press releases indicate that Unisound U1-OCR achieves formidable benchmarks—such as a score of 95.1 on OmniDocBench V1.5—exhaustive analysis confirms that it operates primarily as a commercial, closed-source solution [16, 17, 18]. There is no publicly accessible GitHub repository containing its model weights, nor does it possess a permissible open-source license enabling unrestricted local deployment on edge hardware [19, 20].

To satisfy the strict open-source requirement without compromising state-of-the-art extraction accuracy, the architecture must pivot entirely. The optimal, empirically validated alternative is **GLM-OCR** (zai-org/GLM-OCR) [21, 22].

GLM-OCR is a multimodal optical character recognition model built upon the GLM-V encoder-decoder foundation [21, 23]. It is fully open-source, with the model weights released under the permissive MIT License and the complete inference pipeline code licensed under Apache 2.0 [21, 22]. GLM-OCR is specifically engineered to understand complex document layouts, preserving the semantic structure of headings, nested tables, mathematical formulas, and diverse fonts, directly addressing the dense, chaotic nature of Indian corporate tax filings and annual reports [21, 23].

The architectural mechanics of GLM-OCR are exceptionally suited for a 24 GB VRAM budget. Despite achieving a score of 94.62 on OmniDocBench V1.5—statistically tying with the proprietary U1-OCR and outperforming massive Vision-Language Models (VLMs) like Gemini-3-Pro—GLM-OCR operates with a mere 0.9 Billion parameters [21, 22]. At bfloat16 (BF16) precision, the model requires less than 3 GB of VRAM during active inference, leaving massive computational overhead for the downstream generative reasoning models [24].

This efficiency is achieved through a highly optimized dual-component architecture. First, the CogViT visual encoder—a Vision Transformer pre-trained on large-scale image-text data—produces dense visual tokens encoding the layout, text regions, and structural anomalies of the document [21, 25]. These tokens are routed through a lightweight cross-modal connector utilizing token downsampling, which drastically compresses the visual sequence length before it reaches the language decoder, lowering compute costs exponentially [25]. Finally, the ~0.5B parameter GLM language decoder generates text tokens conditioned on both visual and textual contexts [25]. Furthermore, GLM-OCR introduces Multi-Token Prediction (MTP) loss and stable full-task collaborative reinforcement learning to dramatically improve training efficiency and mitigate spatial hallucinations [21].

For complex chart extraction and tabular disambiguation, the pipeline avoids proprietary black-box APIs like Reducto, which, despite high claims of accuracy on their RD-TableBench, lock data behind paywalls and cloud dependencies [26, 27]. Instead, GLM-OCR is utilized to generate structure-first outputs directly into semantic Markdown or LaTeX formats [23]. Because the model is exceptionally compact, it can be seamlessly fine-tuned for specific, highly degraded Indian banking document formats using the open-source LLaMA-Factory framework [21].

The fine-tuning dynamics of GLM-OCR are highly favorable for local workstations. Full-parameter fine-tuning, which updates both the visual encoder and the language decoder, requires approximately 24 GB of VRAM, perfectly matching the maximum capacity of an RTX 4090 [21, 28]. Alternatively, Parameter-Efficient Fine-Tuning (PEFT) via Low-Rank Adaptation (LoRA) consumes only 8 GB of VRAM [21, 28]. This allows credit institutions to continuously update the OCR model on new, obscure document types locally without requiring massive GPU clusters. In deployment, GLM-OCR achieves a sustained throughput of 1.86 PDF pages per second and 0.67 images per second under single replica concurrency, outperforming comparable local models and ensuring that 100-page annual reports are ingested and

converted to structured Markdown in under a minute [22].

Pillar III: The Cognitive Core and Distributed Reasoning

The extraction of raw, structured text via GLM-OCR is only the initial ingestion phase. The decisioning engine must actively reason over financial covenants, assess macroeconomic sentiment, and evaluate the "Five Cs" of creditworthiness (Character, Capacity, Capital, Collateral, Conditions). Running a frontier-level reasoning model locally presents a severe computational bottleneck. A standard 70 Billion parameter model executing in FP16 precision requires upwards of 140 GB of VRAM, rendering it impossible to deploy on a single edge GPU [29].

To operate securely within the 24 GB VRAM boundary, the architecture leverages advanced knowledge distillation and rigorous quantization techniques, deploying **DeepSeek-R1-Distill-Llama-8B** as the primary cognitive engine [30].

DeepSeek-R1 represents a watershed moment in open-source artificial intelligence, demonstrating logical deduction and reinforcement learning capabilities parallel to proprietary models like OpenAI's o1 [31, 32]. The distilled 8B variant is created by taking the highly complex Chain-of-Thought (CoT) reasoning trajectories generated by the massive 671 Billion parameter DeepSeek-R1 teacher model and training them into the highly efficient Llama-3 8B architecture [30]. This process compresses the deep deductive logic of a supercomputing cluster into a footprint viable for consumer hardware [30].

To physically fit the 8B model into the VRAM budget alongside GLM-OCR and the operating system overhead, the model weights are aggressively quantized using the GGUF format [2]. The system specifically utilizes the Q4_K_M quantization profile, which reduces 16-bit floating-point weights to 4-bit integers utilizing block-wise scaling [2]. This specific quantization matrix optimally balances inference quality with memory reduction, shrinking the physical model size by approximately 75% and requiring only ~6 GB of VRAM to load the weights [2]. Allocating an additional 8 to 10 GB of VRAM specifically for the Key-Value (KV) Cache—optimized via mechanisms like FlashAttention—enables the model to maintain a massive context window [33]. This expanded context window is critical, as it allows the model to ingest the entirety of the lengthy, multi-page financial Markdowns output by GLM-OCR in a single pass without spilling over into the vastly slower system RAM, which would catastrophically degrade tokens-per-second throughput [33].

Financial Domain Adaptation via LoRA Swapping

While DeepSeek-R1-8B acts as a remarkably powerful generalist reasoning engine, corporate credit appraisal requires hyper-specialized domain knowledge regarding specific regulatory frameworks, financial ratios, and accounting standards. Conducting standard full-parameter

fine-tuning for each specific task is computationally wasteful and practically impossible on a single GPU. Instead, the architecture utilizes Low-Rank Adaptation (LoRA) adapters to dynamically inject expertise.

LoRA drastically reduces the number of trainable parameters by freezing the pre-trained model weights and injecting trainable rank decomposition matrices into each layer of the Transformer architecture [30, 34]. The resulting adapter files are exceptionally small, typically occupying less than 100 MB of storage [35]. Open-source adapters, such as DeepSeek-R1-Distill-Llama-8B-finance-v1—which is fine-tuned on the Finance-Instruct-500k dataset—imbue the base model with specific capabilities for financial question answering, covenant breach detection, and macroeconomic sentiment analysis [36, 37].

These micro-adapters reside passively in the 64 GB DDR5 system memory. During execution, the orchestration layer dynamically swaps them into the GPU VRAM at inference time based on the active task [35]. For example, when the pipeline transitions from reading a balance sheet (requiring the Financial Adapter) to analyzing a complex legal notice regarding promoter litigation, the orchestrator seamlessly unloads the financial matrices and injects a legal-reasoning adapter. This effectively multiplexes the 24 GB VRAM, allowing a single 8B model to mimic a much larger, multi-expert ecosystem without ever exceeding hardware limits.

Entity Resolution: Crocodile over TabLinkLLM

A critical point of failure in automated credit appraisal is Entity Resolution—the ability to accurately distinguish between individuals or subsidiaries sharing similar names within dense data structures. The original architecture specified the use of TabLinkLLM for linking entities within tabular data to knowledge graphs [1]. However, despite mentions in academic literature, explicit, usable source code repositories for TabLinkLLM remain obscured within academic supplements, presenting a severe risk to long-term maintainability and strictly violating the open-source directive [38, 39].

To enforce the 100% open-source mandate, the architecture pivots to **Crocodile**, an Apache 2.0 licensed Python library explicitly designed for efficient entity linking over tabular data [40, 41]. Developed by the enRichMyData project, Crocodile seamlessly integrates with standard data processing pipelines utilizing Pandas dataframes [41]. It bypasses the need for massive LLMs for basic linkage, instead utilizing scalable deep neural network architectures and ontology-based candidate lookups to perform high-fidelity semantic table interpretation [40]. By operating locally and linking extracted subsidiary names directly to established internal knowledge bases, Crocodile ensures that risk scores are driven by absolute entity truth without relying on rate-limited, external API calls.

Pillar IV: Graph-Theoretic Fraud Detection via the FLAG Framework

Corporate lending fraud, particularly circular trading within GST networks, is fundamentally a topological problem. Fraud rings and synthetic identity clusters artificially inflate turnover to manipulate credit limits by embedding illicit transactions within massive volumes of legitimate noise [1]. Traditional, independent, and identically distributed (i.i.d.) machine learning models—such as Random Forests or Isolation Forests—fail to detect these typologies because they evaluate individual rows of data while ignoring the relational network topology entirely [1, 42].

The architecture addresses this strictly through Heterogeneous Graph Neural Networks (GNNs), specifically implementing the open-source **FLAG (Fraud Detection with LLM-enhanced Graph Neural Network)** framework introduced at KDD 2025 [43, 44, 45]. The FLAG framework elegantly solves the pervasive "neighborhood camouflage" problem, wherein fraudulent entities deliberately transact with highly reputable nodes (e.g., paying utility bills or transacting with blue-chip corporations) to mathematically disguise their illicit nature within the transaction graph [1, 46].

FLAG operates via a highly sophisticated dual-pronged methodology to defeat this camouflage:

1. **LLM-based Node Enhancement:** The crucial innovation of FLAG is utilizing a Large Language Model to extract highly discriminative textual features from transaction descriptions, entity names, and associated metadata before routing them into the graph [1, 46]. In this local architecture, the quantized DeepSeek-R1-8B model processes the raw transaction metadata, generating dense vector embeddings that represent the semantic intent of the cash flow. These embeddings become the initial node features.
2. **Semantic Similarity Neighbor Sampling:** FLAG does not indiscriminately aggregate features from all adjacent nodes. It selectively filters out the noise of camouflaged neighbors by exclusively sampling adjacent nodes that exhibit high semantic relevance to the target node [1, 44]. This radically reduces the input space, prevents computational explosion, and completely mitigates the smoothing effect of deceptive, high-reputation links [1].

To prevent the phenomenon of over-smoothing—where node representations become mathematically indistinguishable after multiple network layers—the system utilizes a Jump-Attentive GNN (JA-GNN) architecture. The JA-GNN incorporates skip connections that preserve raw features from earlier layers across multi-hop distances. Mathematically, the representation of node i at layer L is formulated as:

$$h_i^{(L)} = COMBINE(h_i^{(0)}, h_i^{(1)}, ..., h_i^{(L)})$$

This formulation guarantees that a highly suspicious signal—such as a shared director linked to an insolvent shell company—located three hops away in the graph is not mathematically diluted

by intervening legitimate transactions [1].

Crucially, the entire graph construction and GNN processing executes purely in the 64 GB DDR5 system memory. By leveraging libraries such as PyTorch Geometric or DGL, and computing centrality metrics (e.g., PageRank, FastRP) on the CPU, the system identifies "Star Sellers" and "Chain Motifs" without taxing the GPU VRAM, reserving the GPU strictly for the LLM semantic embedding generation [1, 47].

Pillar V: Swarm-Intelligence and Web Extraction

The role of the "Digital Credit Manager" requires the autonomous, web-scale gathering of secondary research. This entails crawling the web for news related to company promoters, sector-specific headwinds, and litigation histories across highly fragmented sources [1]. To achieve this, the architecture relies on a distributed network of intelligent agents operating with autonomy, coordination, and stateful memory.

Orchestration: The Microsoft Agent Framework

The multi-agent orchestration layer requires a robust framework capable of handling complex, non-linear workflows. While platforms like LangGraph treat agent behavior as an advanced state machine and are highly popular for Python prototypes [48, 49], the architecture explicitly adopts the **Microsoft Agent Framework (MAF)** [50, 51]. Released entirely open-source under the MIT license, MAF represents the structural convergence of AutoGen's experimental multi-agent orchestration patterns and Semantic Kernel's enterprise-grade stability and type safety [52, 53].

MAF is demonstrably superior for an edge-optimized, production-grade credit architecture due to three intrinsic capabilities:

1. **Durable State Persistence:** Unlike lightweight loops, MAF distinguishes ordinary in-process agents from "durable agents" [48]. A durable agent persists its execution state, conversation history, and tool-call ledger to external storage (e.g., the local DuckDB instance). If the local edge hardware experiences a timeout, VRAM bottleneck, or restart during a multi-hour crawl of the e-Courts portal, the agent recovers its exact context without losing progress or requiring expensive LLM token re-computation [48, 54].
2. **Native Model Context Protocol (MCP):** MAF integrates the open-standard MCP directly out of the box [55, 56]. MCP acts as a universal $N \times M$ translator, allowing the DeepSeek-R1 powered agents to securely interface with the local DuckDB lakehouse, external scraping tools, and internal risk metrics without requiring the development of fragile, custom point-to-point APIs [1, 52].
3. **Local Model Abstraction:** Through Microsoft.Extensions.AI.IChatClient (or its Python SDK equivalent), the framework interacts with local models running on llama.cpp or Ollama indistinguishably from cloud APIs [57, 58]. This abstraction allows the orchestration logic to dynamically request the unloading and loading of LoRA adapters transparently, managing

the 24 GB VRAM constraint without hardcoding hardware logic into the agent's behavior [57].

Utilizing MAF, the system deploys a "Magentic Orchestration" pattern [56]. A central Manager Agent delegates tasks concurrently to a Web Research Agent (interfacing with scraping tools), a Data Agent (executing SQL on DuckDB), and a Risk Agent (analyzing FLAG graph topologies), synthesizing their findings into a shared, persistent session context [55, 56].

Stealth Web Extraction: Self-Hosted Firecrawl

To conduct deep secondary research, the Web Research Agent must autonomously navigate modern websites protected by highly sophisticated anti-bot systems, JavaScript rendering walls, and CAPTCHAs. Standard Python scripts (e.g., BeautifulSoup) are entirely insufficient [1]. The architecture integrates **Firecrawl**, an advanced API capable of rendering dynamic content, managing proxy rotation, and extracting structured Document Object Model (DOM) data into clean, LLM-ready Markdown [59].

Critically, Firecrawl is open-source and capable of being fully self-hosted via Docker Compose, eliminating reliance on their cloud SaaS offering [60]. By deploying the firecrawl-simple fork—which is explicitly optimized for self-hosting by stripping out proprietary billing logic, Stripe integrations, and cloud telemetry—the system ensures a lightweight footprint [61]. Operating a local Firecrawl instance requires approximately 4 GB of system RAM and 2 CPU cores, fitting comfortably alongside the DuckDB instance within the host's 64 GB DDR5 limit [62, 63].

Orchestration Capability	**LangGraph (Python Ecosystem)**	**Microsoft Agent Framework (MIT License)**
Control Flow Paradigm	State Machine and Graphs [48]	Magentic, Sequential, Handoff, Group Chat [56]
State Persistence	Checkpoint memory buffers [49]	Durable agents with persistent external storage [48]
Tool Integration	Custom tool bindings	Native Model Context Protocol (MCP) clients [55]
Enterprise Readiness	Requires LangSmith for deep observability [49]	Built-in OpenTelemetry, Semantic Kernel heritage [52]

Pillar VI: Neuro-Symbolic Decisioning and Regulatory Reality

The final output of the Intelli-Credit challenge is the Recommendation Engine, which must seamlessly synthesize all ingested structured data, network topologies, and swarm insights to generate a Credit Appraisal Memo (CAM). Crucially, the engine must utilize a transparent, explainable scoring model, explicitly detailing why a limit was suggested or rejected [1].

In the highly regulated financial sector, relying exclusively on Generative LLMs for final risk adjustments is an unacceptable compliance failure. LLMs excel at pattern recognition but lack deterministic mathematical reasoning traces, rendering them vulnerable to hallucinations [1]. To fulfill the Reserve Bank of India's (RBI) mandates for absolute transparency and algorithmic auditability, the decisioning engine culminates in a **Neuro-Symbolic AI** framework [22, 64]. Neuro-Symbolic AI bridges the high-dimensional, adaptive pattern-recognition of neural networks with the rigorous, rule-based, deterministic logic of symbolic reasoning [1].

This logic layer is constructed using two fully open-source, synergistic systems:

PyReason: Open-World Temporal Logic

Licensed under the BSD 3-Clause license, **PyReason** is an advanced inference software optimized for generalized annotated logic and open-world temporal reasoning [65, 66]. PyReason performs deductive reasoning directly over graph-based data structures, enabling it to natively ingest the topological fraud outputs from the FLAG framework [67].

Traditional logic solvers fail in real-world scenarios due to the "closed-world assumption"—they assume any fact not explicitly known to be true is false. PyReason circumvents this by operating on a lower semi-lattice structure, allowing it to gracefully handle uncertainty and incomplete knowledge (e.g., missing financial records) without halting execution or requiring exhaustive computational Skolemization [68]. Furthermore, PyReason is exceptionally memory efficient. Empirical evaluations demonstrate that it scales sub-linearly, running inference over graphs with millions of nodes in milliseconds while utilizing a fraction of system RAM, fitting effortlessly into the 64 GB limit [68, 69]. Most importantly, it generates fully human-readable, deterministic "chains-of-thought," directly linking outcomes to specific inputs [67].

IBM Logical Neural Networks (LNN)

Developed by IBM and open-sourced under the Apache 2.0 license, **Logical Neural Networks (LNN)** enforce a strict 1-to-1 correspondence between neural network nodes and the elements of weighted real-valued logic formulae [70, 71]. Unlike standard neural networks, which only process data in a feed-forward manner, inference in an LNN is omnidirectional [70]. It functions as a classical first-order logic theorem prover capable of gradient descent [70].

When the DeepSeek-R1 model processes a Credit Officer's qualitative input (e.g., "Factory machinery appears obsolete") [1], it updates the probability matrix for the borrower's "Capacity" score. Simultaneously, the LNN evaluates hard constraints derived from the latest RBI Master Directions (e.g., maximum Co-Lending exposure caps or priority sector lending minimums). The integration of this symbolic knowledge is mathematically enforced via a penalized loss function during the model's operation:

$$L_{Total} = L_{NN}(p) + \lambda \sum_{k \in K} w_k^2$$

Here, $L_{NN}(p)$ represents the standard predictive loss of the neural network given the probability distribution P, while $\lambda \sum w_k^2$ acts as a regularization penalty ensuring that the learned weights strictly conform to the symbolic constraints defined in the regulatory set K [1, 70]. The LNN actively minimizes logical contradiction, yielding absolute resilience against inconsistent knowledge [71].

By grounding the generative outputs in PyReason and LNNs, the CAM Generator guarantees zero hallucinations. If the system recommends a rejection, it utilizes the neuro-symbolic traceability to generate highly specific rationales. For example: "Rejected due to high litigation risk associated with Promoter X identified via Firecrawl secondary research, and a 3-hop circular trading chain motif identified via FLAG topology, explicitly violating RBI exposure clause 4.2."

Systems Integration: The Hardware Feasibility Matrix

To validate the viability of this localized, 100% open-source architecture, hardware resource allocation must be rigorously mapped against the constrained 24 GB VRAM and 64 GB DDR5 RAM budget.

System Memory (DDR5 64 GB) Allocation:

- **Host OS & Essential Background Services:** ~4 GB.
- **Data Substrate (DuckDB & LanceDB):** ~16 GB. By utilizing vectorized execution and zero-copy Arrow memory mapping, the lakehouse scales dynamically, keeping active querying highly localized within the CPU cache without massive memory overhead [7, 10].
- **Web Extraction Engine (Firecrawl Docker):** ~4 GB [63].
- **Graph Processing (FLAG / PyTorch Geometric):** ~10 GB. Topology calculations (PageRank, node degree) are executed in CPU memory before embeddings are generated [1, 72].
- **Neuro-Symbolic Logic Space (PyReason & LNN):** ~8 GB. The memory footprint of PyReason scales sub-linearly with graph size [69].

- **Passive LoRA Storage:** ~1 GB (holding multiple ~100 MB adapters for instant swapping) [35].
- **Total System RAM Utilized:** ~43 GB, leaving a highly secure ~21 GB headroom for operating system safety limits and deep context caching.

GPU Memory (24 GB VRAM) Allocation:

- **Visual Document Understanding (GLM-OCR 0.9B):** Executing in BF16 precision, the vision-language parsing requires roughly 2.5 GB to 3.0 GB of VRAM during active extraction batches [21, 24].
- **Cognitive Core (DeepSeek-R1-Distill-Llama-8B):** Utilizing Q4_K_M GGUF quantization, the base model weights consume approximately 6.0 GB of VRAM [2].
- **Active LoRA Adapter:** ~0.1 GB (dynamically swapped into memory by the Microsoft Agent Framework orchestrator based on the active task) [35].
- **Context Window (KV Cache):** The remaining ~15 GB of VRAM is strictly dedicated to the Key-Value cache [33]. Utilizing PagedAttention and efficient batching, this massive allocation prevents the LLM from spilling context over to the slower system RAM, allowing it to process massive Markdown documents generated by GLM-OCR at maximum token generation speed [33].

Conclusion

The Intelli-Credit architecture detailed herein definitively solves the "Data Paradox" without relying on a single proprietary, closed-source, or cloud-hosted component. By systematically deconstructing heavy enterprise dependencies—replacing Databricks with the DuckDB/LanceDB lakehouse, substituting proprietary vision APIs with GLM-OCR, and avoiding massive LLMs via a heavily quantized, LoRA-adapted DeepSeek-R1-8B model—the system operates flawlessly within a 24 GB VRAM and 64 GB RAM envelope.

The integration of the FLAG framework mitigates complex topological fraud, while the Microsoft Agent Framework provides the durable orchestration necessary for autonomous, stealth-enabled secondary research via Firecrawl. Finally, by anchoring the generative outputs in the mathematically rigorous, formal logic of IBM's Logical Neural Networks and PyReason, the engine ensures that the resulting Credit Appraisal Memos are fully transparent, completely auditable, and strictly compliant with regulatory standards. This paradigm represents the apex of localized, high-security, open-source financial technology.